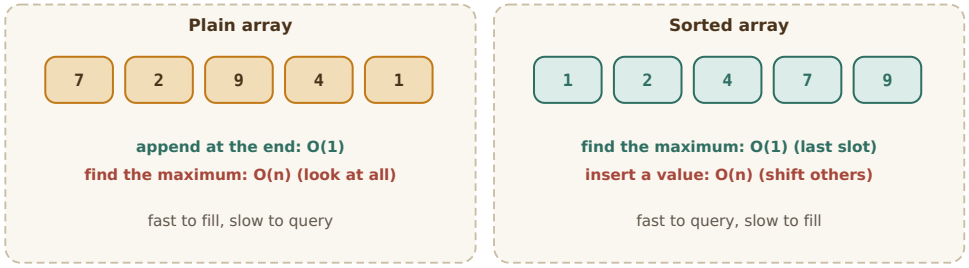
CORE JAVA SERIES — DAY 15

Collections Framework, Iterable & Iterator

Why different data structures exist, what each one is good at, how Java gathers them under one Collection hierarchy — then the Iterable / Iterator pair: how every collection can be walked element by element, how it works inside, how to write your own, and why the design is split in two.

Why There Are Many Data Structures

A data structure is a way of storing data so that certain operations become cheap. No single structure is cheap at everything — each one trades speed in one operation for speed in another. Take a plain array: appending a value at the end is **O(1)**, but finding the maximum means inspecting every slot, **O(n)**. Keep the array *sorted* and the maximum is instant (**O(1)**, it is the last slot), but now inserting a value costs **O(n)** because other values must shift.



Same data, opposite strengths: choosing a structure means choosing which operation you want fast.

That is why computer science has a whole family of structures. Below is the core idea of each, followed by the Java class that already implements it, so you never have to build it yourself.

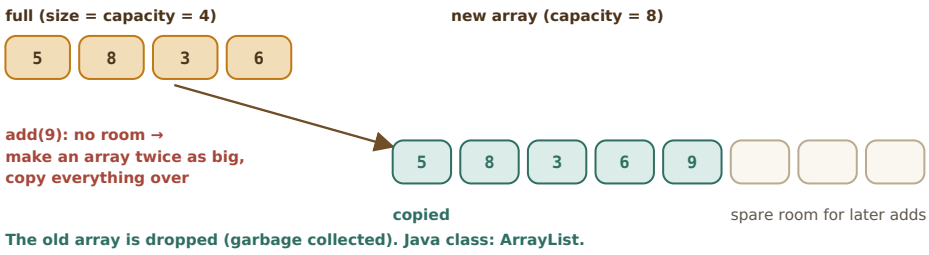
The Building Blocks

Dynamic array → ArrayList

A normal array has a fixed size. A dynamic array hides that: when it becomes full it creates an array **twice as large**, copies everything across and drops the old one. Values sit side by side, so reading by index is instant.

```
class DynamicArray {
    private int[] arr = new int[4];
    private int size = 0;

    void add(int value) {
        if (size == arr.length)                     // full?
            arr = java.util.Arrays.copyOf(arr, size * 2);   // double and copy
        arr[size++] = value;
    }
}
```

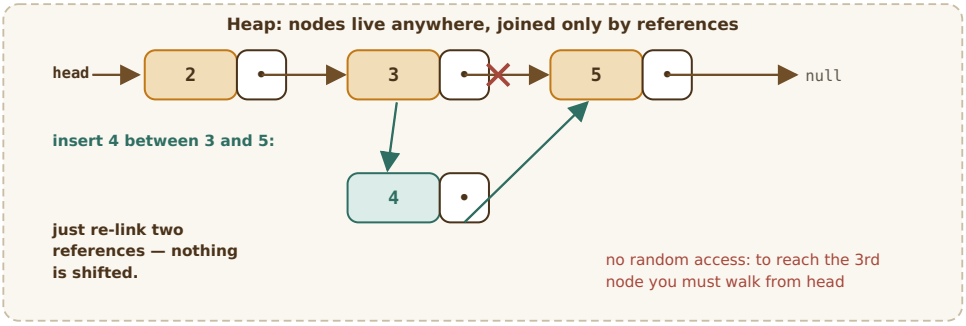


Growing by doubling keeps appends cheap on average.

Linked list → LinkedList

A linked list stores each value in a small object called a **node** that holds the data and a reference to the next node; a `head` reference marks the start. The nodes can be anywhere in memory — only the references join them. Inserting in the middle needs no shifting, just two re-links; the price is that there is no jump-to-index.

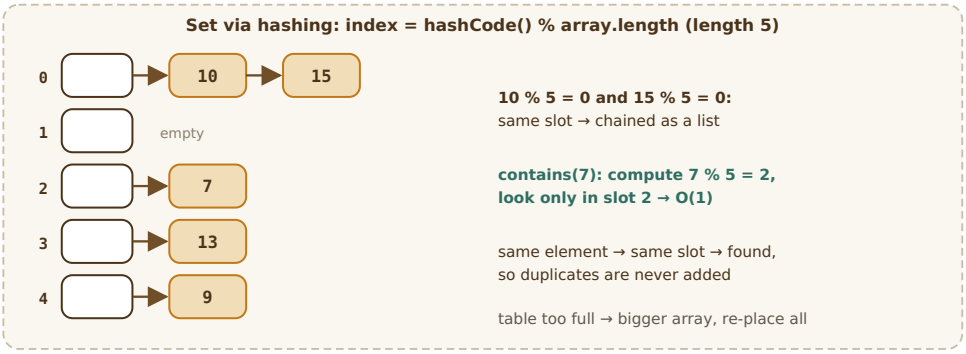
```
class MyLinkedList {
    private static class Node {
        int data;
        Node next;                // reference to the following node
        Node(int data) { this.data = data; }
    }
    private Node head;            // start of the list
}
```



A node stores a value and a reference; inserting only changes references.

Set → HashSet, LinkedHashSet, TreeSet

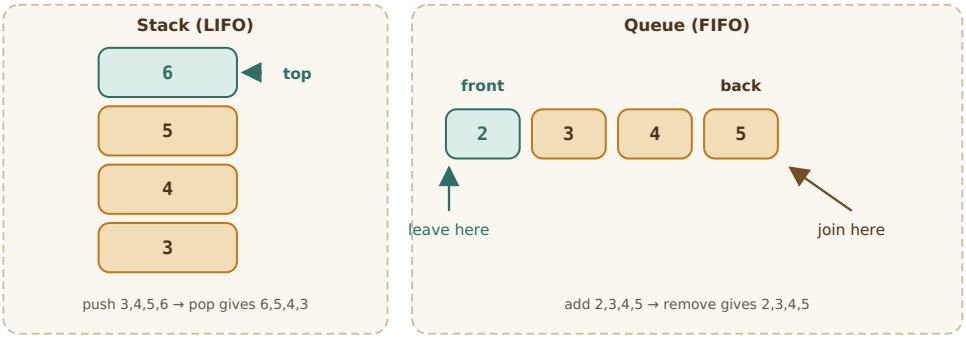
A set answers "is this element present?" very fast and never holds duplicates. The classic way is **hashing**: put an element in the array slot `hashCode() % array.length`. To test membership you compute the same slot and look only there — **O(1)**. If two elements land in one slot they are chained together, and when the table gets crowded it is rebuilt larger. (This is the conceptual picture; Java refines the exact formula.)



Hashing sends each element straight to its slot; collisions form a chain.

Stack and Queue → Stack, ArrayDeque, PriorityQueue

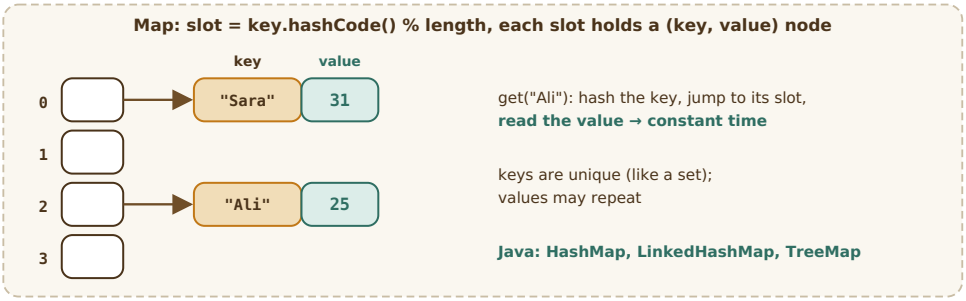
These two are defined by *order of removal*, not by storage. A **stack** is last-in-first-out (a pile of plates: you take the top one). A **queue** is first-in-first-out (a line at a counter). Both are usually built over an array or a linked structure, and neither offers "give me element number i".



Stack removes from the top; queue removes from the front.

Map → HashMap, LinkedHashMap, TreeMap

A map stores **key-value pairs**. Each entry is a node holding a key and a value, placed in the slot given by `key.hashCode() % length`. Looking up by key is constant time, and keys cannot repeat.

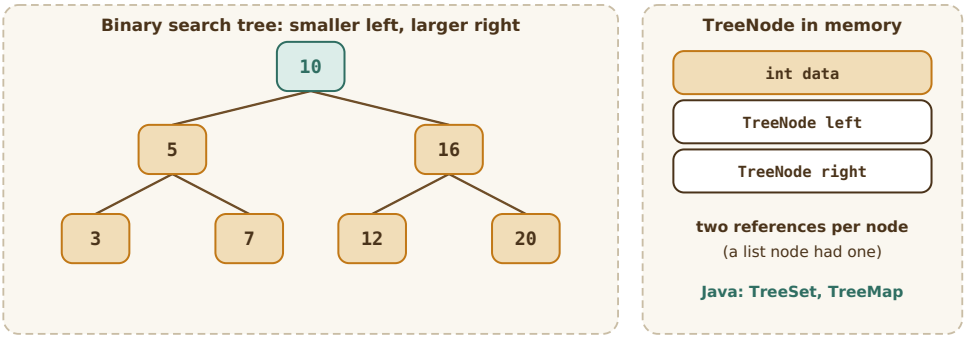


A map is a hash table whose nodes carry a value next to the key.

Tree → TreeSet, TreeMap

A tree stores data hierarchically (folders inside folders are the everyday example). The most useful kind is the **binary search tree (BST)**: each node has at most two children; smaller values go to the left, larger to the right. A tree node therefore needs two references instead of one. Java's `TreeSet` and `TreeMap` are built on a self-balancing BST and keep data sorted.

```
class TreeNode {
    int data;
    TreeNode left;     // smaller values
    TreeNode right;    // larger values
}
```



Every node has two references: one for smaller values, one for larger.

Structure	Core idea	Strong at	Java classes
Dynamic array	values side by side, resize by doubling	access by index	ArrayList
Linked list	nodes joined by references	insert / remove in the middle	LinkedList
Set	hash slot, no duplicates	fast “contains”	HashSet, LinkedHashSet, TreeSet
Stack / Queue	LIFO / FIFO removal	ordered take-out	Stack, ArrayDeque, PriorityQueue
Map	key → value via hash	lookup by key	HashMap, LinkedHashMap, TreeMap
Tree (BST)	smaller left, larger right	sorted data	TreeSet, TreeMap

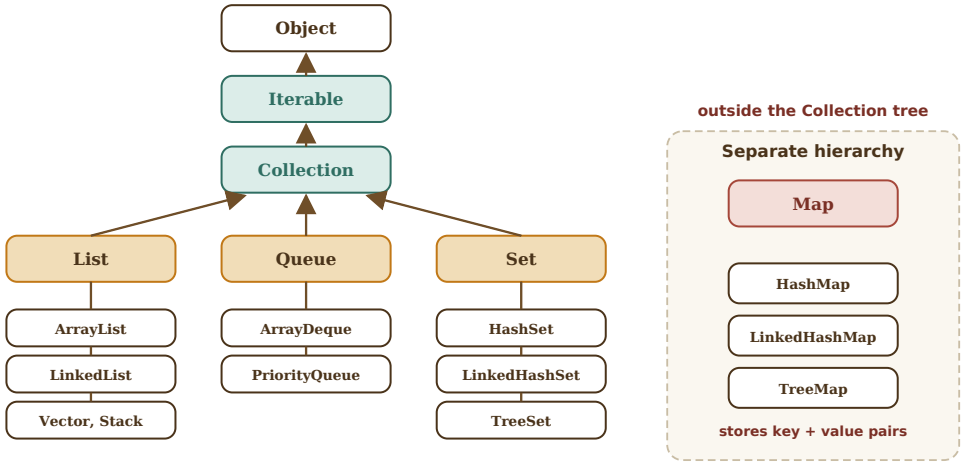
Java Ships Them Ready-Made: the Collections Framework

Java provides all of these as classes, so you rarely build them. But if every class had its own unrelated method names (`add` here, `insert` there), you would have to memorise them all. Java noticed that every one of these is just a **container that holds data**, only organised differently — so it arranged them in one hierarchy and used **inheritance and polymorphism**.

`ArrayList` and `LinkedList` both store a *list* (one continuous, the other scattered), so they share a parent interface `List` that declares add, remove, replace and so on. Each class supplies its own way of doing it, yet the caller writes the same code:

```
List<Integer> l = new ArrayList<>();     // or: new LinkedList<>()
l.add(4);                               // same call, each class does it its own way
```

The same idea goes one level higher. List, Queue and Set are all “data containers”, so Java made them children of one interface, `Collection`, which declares common methods such as `add`. Above `Collection` sits `Iterable` and above that the `Object` class, the parent of every class.



The Collection hierarchy. Map is deliberately a separate tree.

Why Map stands apart: everything under `Collection` deals with single values; a `Map` deals with *key–value pairs*. They are studied together but live in separate hierarchies.

The Problem: Visit Every Element of Any Collection

Suppose you are handed a collection — it might be an ArrayList, a LinkedList, a Set, a Queue or a Stack — and you must visit its elements one by one. The loop you already know is index based:

```
List<Integer> smallList = new ArrayList<>();
smallList.add(2);  smallList.add(3);  smallList.add(5);

for (int i = 0; i < smallList.size(); i++) {
    System.out.println(smallList.get(i));       // 2, 3, 5
}
```

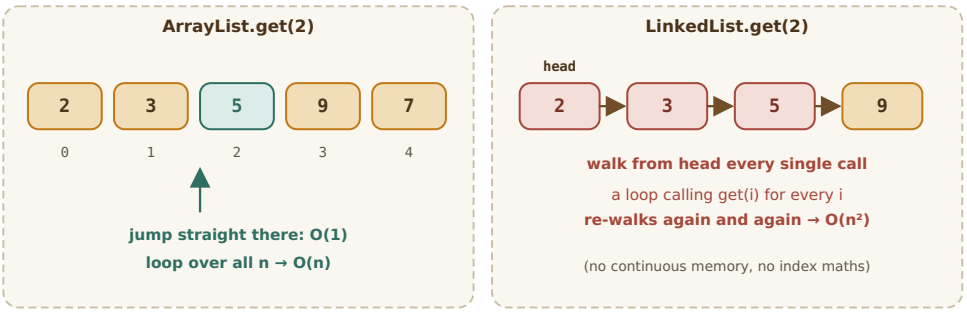
This works for an `ArrayList` because it keeps its values in an array. But it cannot be a general solution, for three separate reasons.

Reason 1 — only Lists have get(i)

Data structures fall into two groups. **Ordered by insertion**: the list family, which hands values back in the order you stored them and offers `get(index)`. **Not ordered by insertion**: sets and maps place elements by hash (you use them to ask "does it exist?" in O(1), not to preserve order), while stacks and queues keep their own order (LIFO / FIFO) and expose only the ends. None of these has `get(i)`, so the index loop simply cannot be written for them.

Reason 2 — even where get(i) exists, it can be terrible

A `LinkedList` is a List, so `get(i)` exists — but its nodes are scattered in memory, so there is no index arithmetic. Every `get(i)` starts again at `head` and hops i times. A loop that calls it for every index repeats that walk each time: **O(n²)**.



Random access jumps; a linked list must walk from head each time.

Reason 3 — draining by hand couples your code to the structure

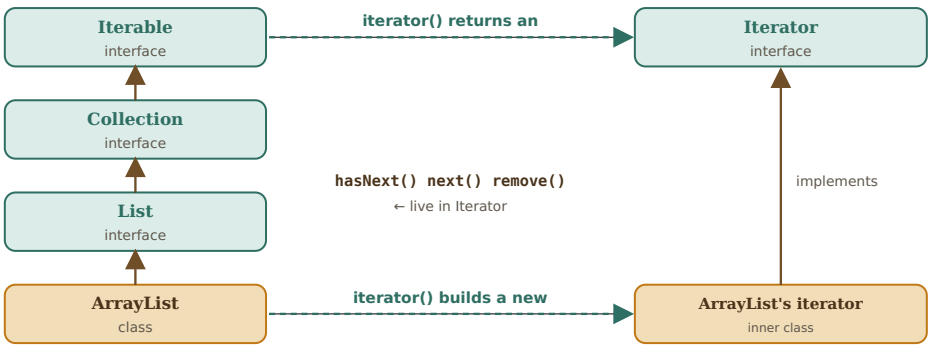
For a stack or queue you could repeatedly take the top / front and remove it until empty. But now your traversal logic is welded to that exact structure, and you must know how it works inside. If someone later swaps `new LinkedList<>()` for `new HashSet<>()`, that logic breaks and has to be rewritten.

What we want: one object we can ask "is there another element?" and "give me the next one", without caring how the data is stored. That is exactly what Java's `Iterable` / `Iterator` pair provides — a textbook case of **abstraction**.

Iterable and Iterator

`Iterable` is the **root interface** for any object whose elements can be traversed one by one. Any collection that wants to be walkable implements it, and that is why it sits at the very top of the hierarchy, above `Collection`. Its key method is `iterator()`, which returns an `Iterator` — a separate interface describing the object that actually does the walking.

An `Iterator` has two essential methods: `hasNext()` answers "is there another element?" (true / false) and `next()` returns that element *and moves one step forward*. Both use generics: `Iterator<E>` returns type `E`.



Iterable sits above Collection; each concrete collection builds its own Iterator.

Checking the real hierarchy confirms it: `ArrayList` extends `AbstractList`, which extends `AbstractCollection`, which implements `Collection`, which extends `Iterable`. So `ArrayList` implements `Iterable` indirectly and must provide a body for `iterator()`.

Using it

```
List<Integer> list = new ArrayList<>();
list.add(10); list.add(20); list.add(30); list.add(40); list.add(50);

Iterator<Integer> it = list.iterator();     // import java.util.*;
while (it.hasNext()) {
    System.out.println(it.next());          // 10 20 30 40 50
}
```

Now the payoff. Change only the first line and the loop stays untouched. Even a `Collection` reference works, because `Collection` extends `Iterable`:

```
Collection<Integer> c = new LinkedList<>();   // or HashSet, ArrayDeque, TreeSet ...
c.add(10); c.add(20); c.add(30); c.add(40); c.add(50);

Iterator<Integer> it = c.iterator();
while (it.hasNext()) System.out.println(it.next());   // loop code never changes
```

Underlying structure	Output order	Why
ArrayList / LinkedList	10 20 30 40 50	insertion order is kept
HashSet	50 20 40 10 30 (one run)	placed by hash; order is unspecified and need not match insertion
ArrayDeque (queue)	10 20 30 40 50	FIFO
TreeSet	10 20 30 40 50	BST, visited in sorted order

The point is not the order but that **the same three lines walked every one of them**; the iterator knows how to move through its own structure so you do not have to.

How It Works Inside

`Iterator` is an interface, so you can never create an object of it directly; some class must implement it. Each collection therefore ships a small private class (an **inner class**) that implements `Iterator`, and its `iterator()` method simply returns a new object of that class. Making it an inner class is deliberate: it can read the collection's private fields (the array, the size, the head) without any plumbing.

ArrayList: a position cursor

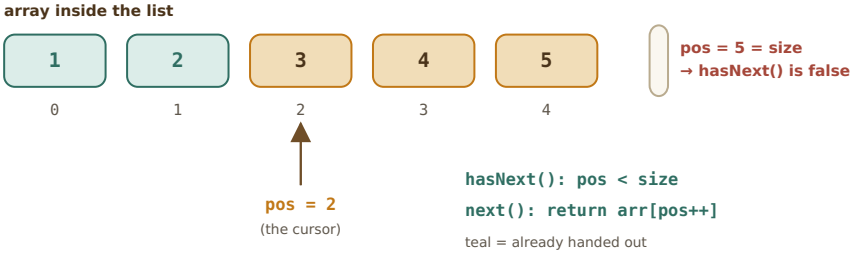
The iterator keeps one number, `pos`, starting at 0. `hasNext()` checks `pos < size`. `next()` returns `arr[pos]` and, thanks to post-increment, moves `pos` forward in the same expression.

```
class MyArrayList implements Iterable<Integer> {
    private Integer[] arr;
    private int size;

    @Override
    public Iterator<Integer> iterator() {
        return new MyArrayListIterator();          // a fresh iterator each call
    }

    private class MyArrayListIterator implements Iterator<Integer> {
        private int pos = 0;                       // the cursor

        @Override public boolean hasNext() { return pos < size; }
        @Override public Integer next()    { return arr[pos++]; }   // value, then move
    }
}
```



The cursor stops moving once it reaches size.

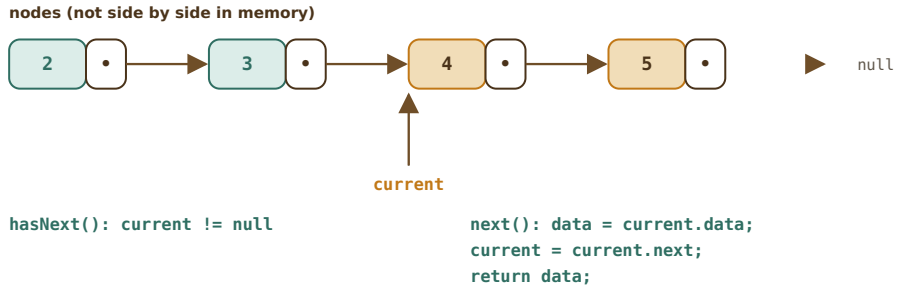
LinkedList: a node reference

There is no array to index, so the iterator keeps a `current` node, starting at `head`. `hasNext()` is "`current` is not null". `next()` saves `current.data`, moves `current` to `current.next` and returns the saved value.

```
private class MyLinkedListIterator implements Iterator<Integer> {
    private Node current = head;                  // start at the first node

    @Override public boolean hasNext() { return current != null; }

    @Override public Integer next() {
        int data = current.data;
        current = current.next;                   // step to the next node
        return data;
    }
}
```



Same two methods, different cursor: a node reference instead of an index.

The other structures follow the same recipe. A hash-based `HashSet` / `HashMap` iterator can walk bucket by bucket and, inside a bucket, chain element by chain element. `TreeSet` / `TreeMap` hand back their BST in **in-order** (sorted) sequence. Stacks and queues built on arrays walk the underlying array. The caller sees none of this.

Writing Your Own Iterable

Let us build a tiny container of names that can be walked. Three steps: implement `Iterable<String>`, override `iterator()`, and return an object implementing `Iterator<String>` with `hasNext()` and `next()`. Since that iterator class is used exactly once, an **anonymous class** is the neat way to write it.

```
import java.util.*;

class NameContainer implements Iterable<String> {
    private String[] names;
    private int size;

    NameContainer(String[] names) {
        this.names = names;
        this.size = names.length;
    }

    @Override
    public Iterator<String> iterator() {
        return new Iterator<String>() {           // anonymous class: used only here
            private int pos = 0;
            @Override public boolean hasNext() { return pos < size; }
            @Override public String next()     { return names[pos++]; }
        };
    }
}

// usage
NameContainer container = new NameContainer(new String[]{"Aditya", "Rohit", "Rohan", "Monu"});
Iterator<String> it = container.iterator();
while (it.hasNext()) System.out.println(it.next());   // Aditya Rohit Rohan Monu
```

The Enhanced for Loop Is Just Sugar

That `iterator()` / `while` code is verbose, so Java gives a shorthand: the **enhanced for loop** (for-each). It is nothing more than *syntactic sugar* — the compiler rewrites it into the iterator loop above, and the JVM only ever sees the long form.

```
for (String name : container) {
    System.out.println(name);                     // same output as the while loop
}
```

The proof: remove `implements Iterable<String>` from `NameContainer` and the loop no longer compiles — *"for-each not applicable to expression type"*. A for-each loop works only on an array or an object that is `Iterable`.

What the Two Interfaces Contain

Interface	Method	What it does
Iterable	`iterator()`	Returns a fresh Iterator over the elements — the one you must implement.
Iterable	`forEach(...)`	Default method taking a lambda; covered later with functional interfaces.
Iterable	`spliterator()`	Advanced; relevant once streams are studied.
Iterator	`hasNext()`	Is there another element? (boolean)
Iterator	`next()`	Returns the next element and advances.
Iterator	`remove()`	Default method: deletes the element last returned by next().
Iterator	`forEachRemaining(...)`	Default method taking a lambda; covered later.

Because `remove()` is a *default* method you only have to implement `hasNext()` and `next()`; `remove()` is optional. Notice there is no `add()`. Adding while walking is ill-defined: in a list, "insert here" has a clear meaning, but in a set the new value may hash to a place the iterator has already passed, so it would never be visited. (Lists do offer a richer `ListIterator` with `add`, met when List is studied.)

```
Iterator<Integer> it = list.iterator();     // [1, 2, 3, 4, 5]
while (it.hasNext()) {
    if (it.next() == 3) it.remove();        // safe removal while walking
}
System.out.println(list);                    // [1, 2, 4, 5]
```

ConcurrentModificationException: Fail-Fast

What if you change the *collection itself* (not through the iterator) while walking it? The iterator notices that the structure changed behind its back and refuses to go on:

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4, 5));
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    int value = it.next();
    if (value == 3) list.remove(Integer.valueOf(value));   // modifies the LIST directly
    System.out.print(value + " ");
}
// prints: 1 2 3   then throws ConcurrentModificationException
```

In a list this may look harmless ("just continue from where you were"), but picture a set: elements can vanish or appear anywhere, so the iterator cannot know whether it already visited something. Rather than guess, it **fails fast** — stops immediately with an exception instead of continuing on shaky assumptions. Safe removal during a walk goes through `it.remove()`, as shown above.

Why Two Interfaces and Not One?

A fair question: why does `ArrayList` not implement `Iterator` itself and define `hasNext()` / `next()` directly? There are two reasons.

1. Separation of concerns

A class should have one responsibility (the Single Responsibility Principle from SOLID). A collection's job is to **store data**; *how to move across that data* is a different job. So Java keeps the movement logic in a separate iterator class, and the collection merely hands one out through `Iterable.iterator()`.

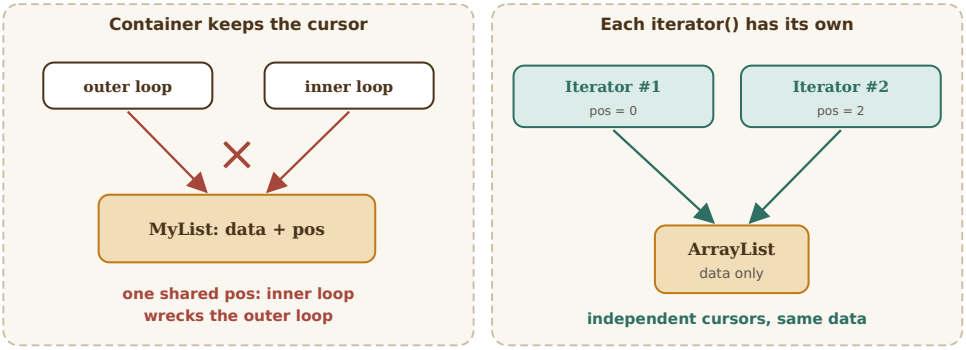
2. The cursor must be independent per walk

This is the deeper reason. Suppose the list itself kept the cursor and implemented `hasNext()` / `next()`:

```
class MyArrayList {                         // NOT how Java does it
    private int[] arr = {10, 20, 30};
    private int size = 3;
    private int pos = 0;                    // one cursor, stored inside the list

    boolean hasNext() { return pos < size; }
    int next()        { return arr[pos++]; }
    void reset()      { pos = 0; }
}
```

First problem: after one full walk `pos` is 3, so a second loop never runs unless you remember to call `reset()`. Second, and fatal: **nested loops**. You want every pair, so an inner loop runs inside an outer loop — but both share the single `pos`. The outer loop reads 10 and advances `pos`; the inner loop then reads 20 and 30 and pushes `pos` to the end; when control returns, the outer loop finds nothing left. Output would stop at `10 20`, `10 30`. Calling `reset()` only wrecks the outer loop's place instead. Adding more cursors by hand would never be enough, since you might nest two, three or four loops.



A cursor inside the container is shared by every loop; a cursor per iterator is not.

With the iterator design the container holds only data, and every call to `iterator()` returns a **brand-new iterator with its own cursor**. Nested loops simply ask for a second one:

```
List<Integer> list = new ArrayList<>(List.of(10, 20, 30));

Iterator<Integer> outer = list.iterator();
while (outer.hasNext()) {
    int a = outer.next();
    Iterator<Integer> inner = list.iterator();     // a separate cursor
    while (inner.hasNext()) {
        System.out.print(a + "-" + inner.next() + " ");
    }
}
// 10-10 10-20 10-30 20-10 20-20 20-30 30-10 30-20 30-30
```

Summary of the design: `Iterable` = “I can be walked” (the collection); `Iterator` = “I am one walk in progress” (the cursor). Each call to `iterator()` starts a new, independent walk, and the walker alone knows how to step through its structure.

Active Recall

Q1. Why is there no single best data structure?

A. Each trades speed in one operation for another: a plain array appends in O(1) but finds the max in O(n); a sorted array finds the max in O(1) but inserts in O(n).

Q2. Why does Java arrange its structures in a hierarchy?

A. They are all containers; a shared parent (List, Collection) gives one method set via inheritance, and polymorphism lets code switch implementations (ArrayList ↔ LinkedList) unchanged.

Q3. Why is Map outside the Collection hierarchy?

A. Collection deals with single values; Map deals with key–value pairs.

Q4. Give three reasons an index loop cannot be the universal way to traverse.

A. Only Lists have get(i); in a LinkedList get(i) walks from the head so a loop is $O(n^2)$; draining by structure-specific methods ties your code to that structure.

Q5. What do Iterable and Iterator each provide?

A. Iterable has iterator(), returning an Iterator. Iterator has hasNext(), next() and an optional remove().

Q6. Why are the iterator classes inner classes?

A. So they can read the collection's private fields (array, size, head) directly.

Q7. What does the compiler turn a for-each loop into?

A. An iterator loop: get iterator(), while hasNext() call next(). That is why the target must be an array or an Iterable.

Q8. What is fail-fast, and how do you remove safely during a walk?

A. If the collection is modified behind the iterator's back it throws ConcurrentModificationException instead of guessing. Remove through it.remove().

Q9. Why does Iterable not simply merge into the collection?

A. Separation of concerns, and each iterator() call returns an independent cursor, which makes nested loops and multiple simultaneous walks possible.